

Building AI Agents using LangGraph.

Want to create multiple AI agents that work together like a research team?

This tutorial shows you how to build one using LangGraph + DuckDuckGo + Llama3.

What We're Building

Goal: An AI Researcher + AI Writer working together.

Agent 1 → Searches the web

Agent 2 → Writes a blog post based on the research

LangGraph → Connects them into a workflow

Output → An AI-generated blog post

Install Dependencies

Make sure you have these installed:

```
pip install langgraph langchain-ollama  
duckduckgo-search langchain-  
community
```

Also, make sure Ollama is installed, and the llama3 model is downloaded.

Define State

LangGraph agents share a state. Here's our data structure:

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END
from langchain_community.tools.ddg_search import
DuckDuckGoSearchRun
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate

class AgentState(TypedDict):
    topic: str
    research_data: List[str]
    blog_post: str
```

Researcher Agent

This agent fetches real-time info from the internet:

```
def researcher(state: AgentState):
    search = DuckDuckGoSearchRun()
    try:
        res = search.run(
            f"key facts and latest news about {state['topic']}"
        )
    except Exception as e:
        res = f"Could not find data: {e}"

    return {"research_data": state.get("research_data", []) + [res]}
```

Writer Agent

This agent turns research into a blog post:

```
prompt = ChatPromptTemplate.from_template("""
You are a tech blog writer.
Write a short, engaging blog post about "{topic}"
using only this research:

{data}

Return only the blog post.
""")

def writer(state: AgentState):
    llm = ChatOllama(model="llama3", temperature=0.7)
    data = state["research_data"][-1] if
state["research_data"] else ""
    res = (prompt | llm).invoke({"topic": state["topic"],
"data": data})
    return {"blog_post": res.content}
```


LangGraph Workflow

Now let's wire everything together:

```
workflow = StateGraph(AgentState)

workflow.add_node("Researcher", researcher)
workflow.add_node("Writer", writer)

workflow.set_entry_point("Researcher")
workflow.add_edge("Researcher", "Writer")
workflow.add_edge("Writer", END)

app = workflow.compile()
```


Run the Multi-Agent System

```
if __name__ == "__main__":
    inputs: AgentState = {
        "topic": "The future of AI Agents",
        "research_data": [],
        "blog_post": "",
    }

    result = app.invoke(inputs)
    print(result["blog_post"])
```